UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Project Report On

AlgoVisualizer Report

SUBMITTED BY:

Name: Ankush Kumar

UID: 25MCA20305

CLASS: MCA

SECTION:25MCA_KAR_1A

SUBMITTED TO:

Prof. Shivam Sharma

Table of Content

S.No.	Content	Page No.
1.	Introduction	04
2.	Background	04
3.	Objective	05
4.	Technology Used	05
5.	Requirement	06
6.	Project Design	07
8.	Output	13
9.	Conclusion	17
10.	Learning Outcomes	18
11.	GitHub Link	18

Acknowledgement

I would like to express my heartfelt gratitude to all those who contributed to the successful development of my project, **"DSA Explorer: Interactive Algorithm Visualizer"**, a full-stack application designed to simplify the understanding of data structures and algorithms through visualization.

First and foremost, I am deeply thankful to my mentors and instructors for their continuous guidance, motivation, and valuable feedback throughout the development of this project. Their insights into algorithm design, system architecture, and full-stack development played a crucial role in shaping the direction and quality of this work.

I would also like to extend my sincere appreciation to my friends and classmates for their constant encouragement, helpful suggestions, and support during the design, implementation, and testing phases. Their feedback significantly improved the usability and functionality of the system.

Lastly, I would like to thank the open-source community and the developers of the technologies used in this project. Tools and frameworks such as React, Node.js, Express, C++, and various documentation resources provided a strong foundation for building an interactive and efficient platform for learning algorithms.

Abstract

The DSA Explorer System is an interactive educational platform developed to enhance the understanding of Data Structures and Algorithms through visualization and real-time execution. Traditional learning methods often make algorithmic concepts difficult to grasp; therefore, this system addresses the problem by integrating graphical representations with structured C++ implementations to provide a clear, step-by-step demonstration of algorithm behavior.

The application is built using a modern full-stack architecture, incorporating React, HTML, and CSS for the frontend, Node.js with Express for backend services, and C++ for implementing core algorithms. It supports a wide range of fundamental topics, including sorting algorithms, graph traversal techniques such as Breadth-First Search (BFS) and Depth-First Search (DFS), and essential data structures like linked lists, stacks, queues, and binary search trees. The backend delivers structured data via APIs, while the frontend dynamically visualizes algorithm execution alongside syntax-highlighted C++ code.

The primary objective of this system is to bridge the gap between theoretical concepts and practical understanding by enabling users to interact with algorithms, observe their execution, and analyze their time and space complexities. By combining visualization with real-time interaction, the DSA Explorer System provides an effective and engaging learning experience.

In conclusion, the project demonstrates the potential of integrating modern web technologies with core programming concepts to create a powerful educational tool that simplifies complex topics and promotes deeper learning.

1. Introduction:

The **DSA Explorer System** is an intelligent and interactive application designed to simplify the understanding of Data Structures and Algorithms through visualization and real-time interaction. It eliminates the difficulty of learning complex algorithmic concepts by combining graphical representations with structured C++ implementations, allowing users to observe how algorithms work step-by-step with clarity and precision.

Developed using a modern full-stack approach, the system integrates **React and HTML/CSS for the user interface, Node.js with Express for backend API handling, and C++ for algorithm implementation**. The application covers fundamental concepts such as sorting algorithms, graph

traversal techniques like BFS and DFS, and core data structures including linked lists, stacks, queues, and binary search trees. The backend provides structured data through APIs, while the frontend dynamically renders visualizations and displays corresponding C++ code using syntax highlighting.

The main objective of the DSA Explorer System is to demonstrate how algorithmic design and data structures can be understood more effectively through visualization and interactive learning. Users can easily select different topics, view their time and space complexities, and analyze the step-by-step execution of algorithms through animations and graphical elements.

Overall, the DSA Explorer System showcases how modern web technologies combined with core programming concepts can be used to build an educational tool that bridges the gap between theory and practical understanding. It serves as both a learning platform and a demonstration of applying data structures and algorithms to interactive software development.

2. Background:

The **DSA Explorer System** was developed to overcome the limitations of traditional methods of learning Data Structures and Algorithms, which often rely on theoretical explanations, textbooks, and static code examples. Understanding algorithms through only written logic can be difficult, time-consuming, and sometimes confusing for students, especially when dealing with complex concepts like graph traversal, sorting techniques, and tree structures. As the complexity of algorithms increases, these traditional learning approaches become less effective and may lead to poor conceptual clarity.

Developed using a full-stack approach, the system uses **React and HTML/CSS for building an interactive user interface, Node.js with Express for backend API handling, and C++ for implementing core algorithm logic**. Additional tools like **Highlight.js** are used for syntax highlighting, making the code more readable and understandable. These technologies work together to simulate real-world algorithm execution in an interactive environment.

Overall, the DSA Explorer System represents a modern educational solution that demonstrates how data structures and algorithms can be effectively taught using visualization and interactive technologies. It enhances conceptual clarity, improves problem-solving skills, and serves as a powerful tool for both academic learning and practical understanding.

3. Objective:

1. **Simplify Algorithm Learning:** Provides an interactive platform to understand complex Data Structures and Algorithms easily through visualization.
2. **Enhance Concept Clarity:** Reduces confusion by showing step-by-step execution of algorithms like sorting, BFS, and DFS instead of only theoretical explanations.
3. **Visualize Algorithm Execution:** Displays algorithms using graphical elements such as bars, nodes, and animations to represent real-time operations.
4. **Implement Core Algorithms:** Includes implementation of important algorithms such as sorting techniques, graph traversal (BFS/DFS), A*, Dijkstra's Algorithm, and operations on data structures like linked lists, stacks, queues, and binary search trees using C++.
5. **Centralize Algorithm Data:** Uses backend APIs (Node.js & Express) to manage and deliver structured information about different algorithms and their complexities.
6. **Educational Application:** Serves as a practical learning tool to demonstrate real-world implementation of Data Structures and Algorithms using modern web technologies.

4. Technology used:

- **C++:** The core programming language used to implement Data Structures and Algorithms such as sorting, graph traversal (BFS/DFS), A*, Dijkstra's Algorithm, linked lists, stacks, queues, and binary search trees, providing high performance and efficiency.
- **React:** Used to build a dynamic and interactive user interface, enabling smooth navigation between different algorithms and real-time updates of content.
- **Node.js:** Acts as the backend runtime environment, handling server-side operations and managing communication between frontend and data.
- **Express.js:** A lightweight web framework used to create REST APIs for delivering algorithm data, descriptions, and C++ code to the frontend.
- **HTML & CSS:** Provide the basic structure and styling of the application, ensuring a clean and responsive layout.
- **Bootstrap:** Used for designing a modern, responsive, and user-friendly interface with pre-built components and styling utilities.
- **JavaScript:** Handles client-side logic, including fetching API data, updating UI dynamically, handling user interactions, and controlling visualizations.
- **Highlight.js:** Used for syntax highlighting of C++ code, improving readability and helping users understand code structure easily.
- **REST API Integration:** Enables communication between frontend and backend using HTTP methods to fetch algorithm-related data dynamically.

- **Visualization Scripts:** Custom JavaScript files (like graph.js, sorting.js, etc.) are used to create animations and graphical representations of algorithm execution.
- **Modular Design:** The project is structured in modules (frontend, backend, visualization scripts), making it scalable and easy to extend with new algorithms and features.
- **JSON Data Handling:** Algorithm details, complexities, and code are managed in structured JSON format and served through APIs.
- **Scalable Architecture:** The system is designed to easily include additional data structures, algorithms, and advanced visualizations in future versions.

5. Requirements:

Hardware Requirements

1. **Computer or Laptop:**
   A system capable of running web applications, a Node.js server, and frontend visualization smoothly.
2. **Processor (CPU):**
   Intel i3 or higher / AMD equivalent (dual-core minimum; quad-core recommended for better performance).
3. **RAM:**
   Minimum 4 GB (recommended 8 GB) to handle frontend rendering, backend processing, and browser execution efficiently.
4. **Storage (HDD/SSD):**
   At least 200 MB of free space for project files, dependencies (Node modules), and source code (SSD recommended for faster performance).
5. **Display:**
   Standard or widescreen monitor for clear viewing of the user interface and algorithm visualizations.
6. **Input Devices:**
   Keyboard and mouse for interacting with the application and navigating between different algorithms.
7. **Graphics Support:**
   Basic graphics support required to render animations and visual elements like graphs, bars, and UI components smoothly.

Software Requirements:

1. **Computer or Laptop:**
   A system capable of running web applications, a Node.js server, and frontend visualization smoothly.

2. **Processor (CPU):**
   Intel i3 or higher / AMD equivalent (dual-core minimum; quad-core recommended for better performance).
3. **RAM:**
   Minimum 4 GB (recommended 8 GB) to handle frontend rendering, backend processing, and browser execution efficiently.
4. **Storage (HDD/SSD):**
   At least 200 MB of free space for project files, dependencies (Node modules), and source code (SSD recommended for faster performance).
5. **Display:**
   Standard or widescreen monitor for clear viewing of the user interface and algorithm visualizations.
6. **Input Devices:**
   Keyboard and mouse for interacting with the application and navigating between different algorithms.
7. **Graphics Support:**
   Basic graphics support required to render animations and visual elements like graphs, bars, and UI components smoothly.

6. Project Design

Architecture:

1. Modular Web-Based Architecture (MWA):

The DSA Explorer System follows a Modular Web-Based Architecture to ensure better organization, scalability, and maintainability. Each module is responsible for a specific task such as frontend visualization, backend API handling, and algorithm representation, ensuring clear separation of concerns and easy future enhancements.

2. The Model:

The Model handles all data and algorithm-related logic in the system. It includes structured information about different Data Structures and Algorithms such as sorting techniques, graph traversal (BFS/DFS), linked lists, stacks, queues, and binary search trees.

- Stores:
  - Algorithm descriptions
  - Time and space complexity
  - C++ implementation code
- Data is managed using JSON format and served through backend APIs.

This acts as the core logic layer of the system.

3. The View:

The View represents the user interface of the application built using React, HTML, CSS, and Bootstrap.

It allows users to:

- Select different algorithms from the sidebar
- View algorithm descriptions and complexities
- See C++ code with syntax highlighting
- Observe visualizations (graphs, sorting animations, etc.)

4. The Controller:

The Controller acts as a bridge between frontend and backend. It is implemented using Node.js and Express.

It:

- Handles user requests (API calls)
- Fetches algorithm data from the Model
- Sends data to the frontend (View)
- Controls flow of data between UI and backend

5. Architecture Benefits:

This architecture ensures that the DSA Explorer System is:

✓ Well-structured and modular
✓ Easy to maintain and extend
✓ Efficient in handling algorithm data
✓ Capable of integrating new algorithms easily
✓ User-friendly with clear visualization

Components

Presentation Layer (Frontend / GUI):

HTML/CSS:

Used to design the interactive user interface of the DSA Explorer system. It allows users to select different algorithms, view descriptions, and interact with visual elements like buttons, panels, and

menus.

- **Bootstrap:**
  Enhances the UI with responsive design, modern components, and styling such as sidebar navigation, cards, and layout grids.
- **JavaScript:**
  Handles dynamic behavior such as fetching API data, updating UI content, handling user
- interactions, and controlling visualization logic.

Business Logic Layer (Algorithmic Processing):

- **Node.js (Core Backend Logic):**
  Handles server-side operations, API creation, and manages communication between frontend
- and data.
- **Express.js Framework:**
  Used to build REST APIs that deliver algorithm data, including descriptions, complexities,
- and C++ code, to the frontend.
- **Algorithm Representation (C++):**
  Includes implementation of algorithms such as sorting, BFS, DFS, linked list operations, stack
- , queue, and binary search tree logic.
- **Dynamic Content Handling:**
  Backend dynamically sends data to the frontend based on user selection of topics.
- **Data Flow Control:**
  Ensures smooth communication between frontend (View) and backend (Model) through
- API requests and responses.

Data Access Layer (Data Management):

- **JSON Data Structure:**
  Acts as the primary data storage format for maintaining algorithm details, including names,
- descriptions, complexities, and C++ code.
- **API Data Handling:**
  Backend uses structured JSON objects (like TOPICS) to store and serve data efficiently.
- **Data Interaction:**
  Supports fetching, updating, and displaying data dynamically through API calls.

Database (Data Structure) Design:

Entities:

Although the DSA Explorer system does not use a traditional SQL database, it follows a structured **JSON-based data model** similar to an Entity-Relationship (ER) design.

- **Topics:**
  - topic_id — Unique identifier for each topic (e.g., sorting, graph, bst)
  - name — Name of the algorithm/data structure
  - category — Type (Arrays, Graphs, Trees, Linear DS)
  - description — Brief explanation of the topic
  - file_name — Associated C++ file name
  - code — C++ implementation of the algorithm
- **Visualization Data:**
  - input_data — User-provided or generated data (array values, graph nodes)
  - steps — Step-by-step execution data for visualization
  - current_state — Current stage of algorithm execution
- **Execution Results:**
  - output — Final result (sorted array, traversal order)
  - comparisons — Number of comparisons performed
  - steps_count — Total steps executed
  - execution_time — Time taken (optional)

Relationships:

• **One-to-Many (Topics → Visualization Data):**
One algorithm/topic can have multiple visualization steps during execution.
• **One-to-One (Topic → Code):**
Each topic has one specific C++ implementation.
• **One-to-One (Execution → Result):**
Each algorithm execution produces one final result.

User Roles

User Role (General User):

• Can select different algorithms or data structures from the interface.
• Can input values (e.g., array for sorting, nodes for graph).
• Can view step-by-step visualization of algorithm execution.
• Can see time and space complexity of selected algorithms.
• Can view and copy C++ implementation code.

Administrator Role (Developer / Analyst):

• Can add or modify algorithm data (JSON structure, code, descriptions).
• Can extend the system by adding new algorithms or data structures.
• Can debug and optimize backend APIs and frontend visualization scripts.
• Can monitor performance and improve UI/UX.

Figure 1: ER Diagram

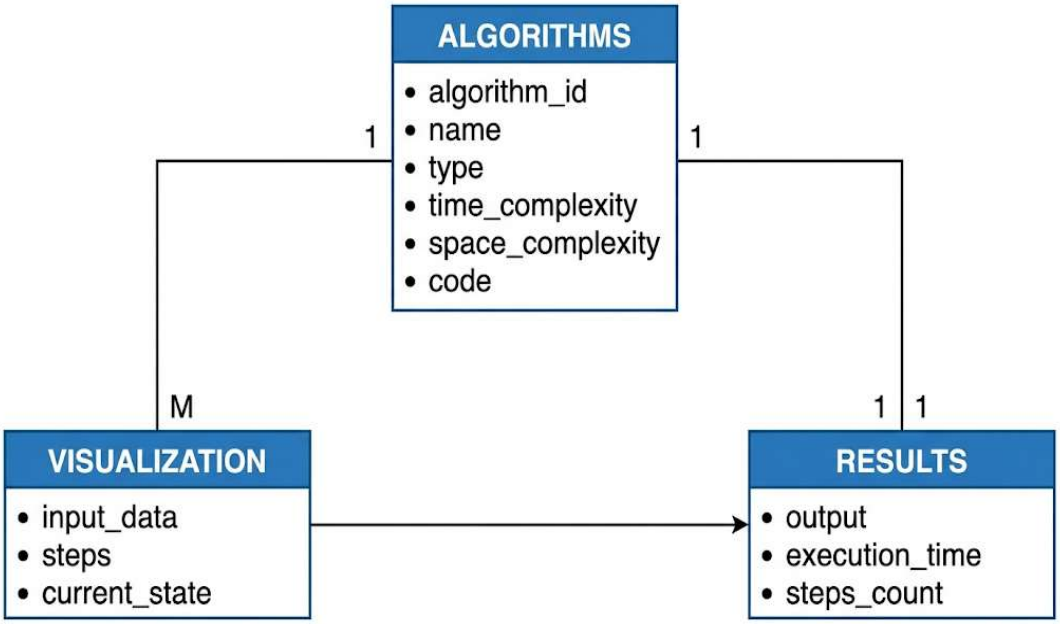
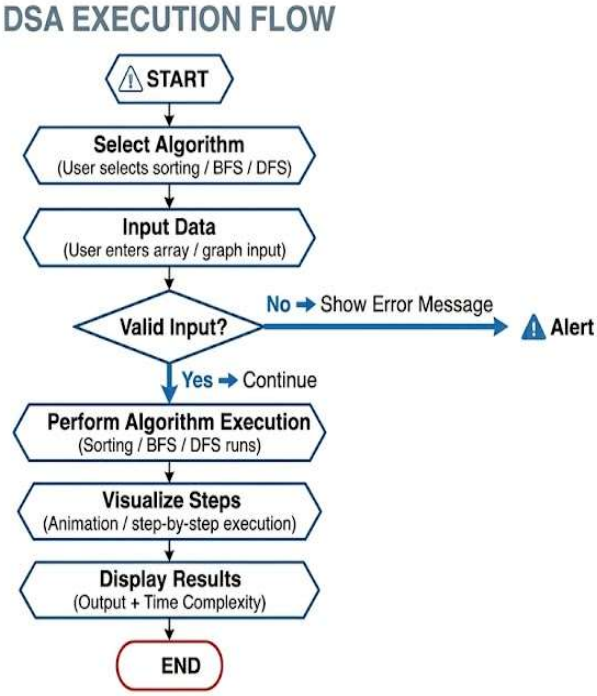


Figure 2: Flow Chart



Output:

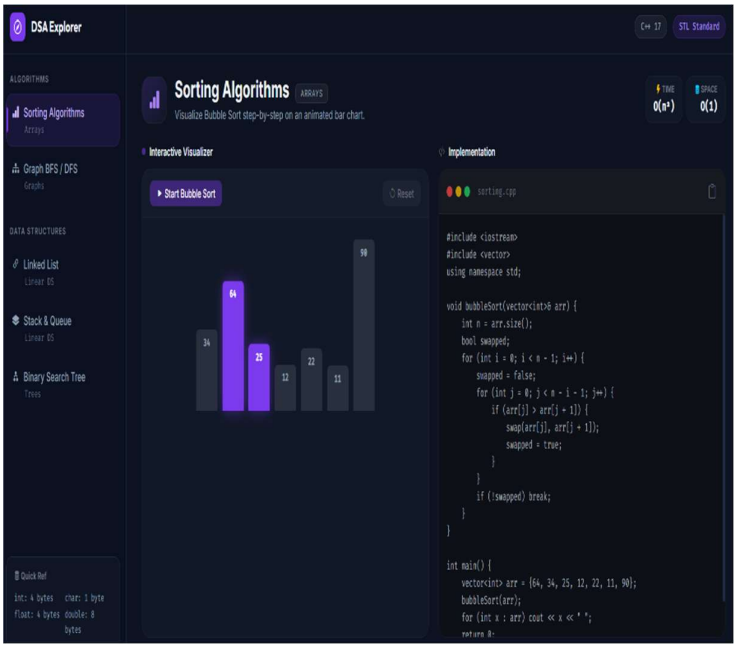


Fig.1 Sorting Visualization

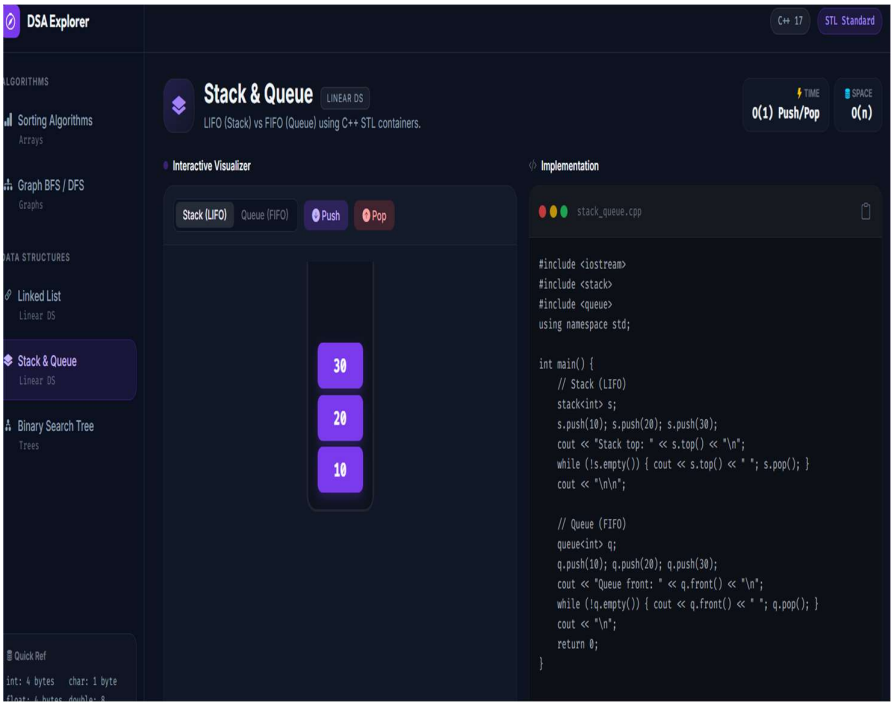


Fig.2- Stack & Queue Visualizer

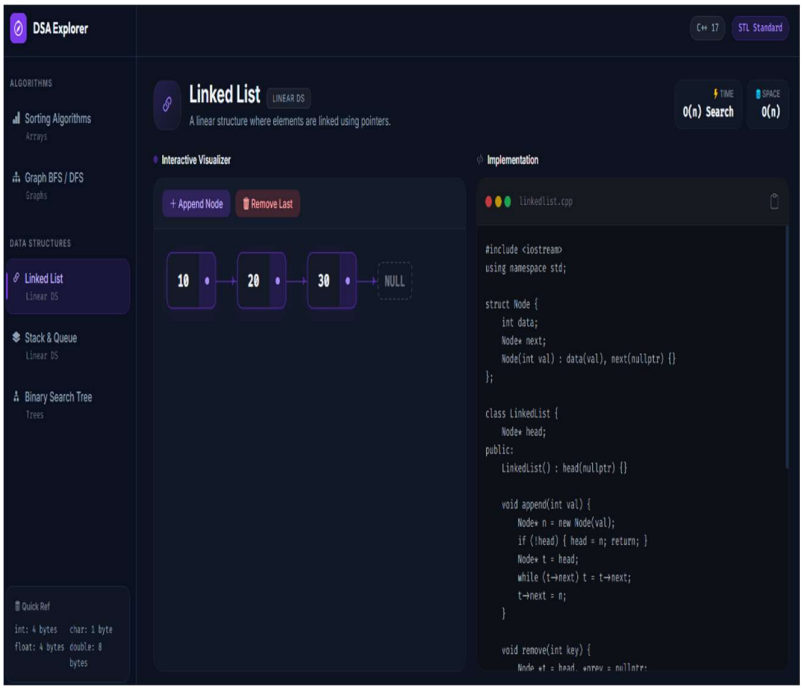


Fig.3 Linked List Visualizer

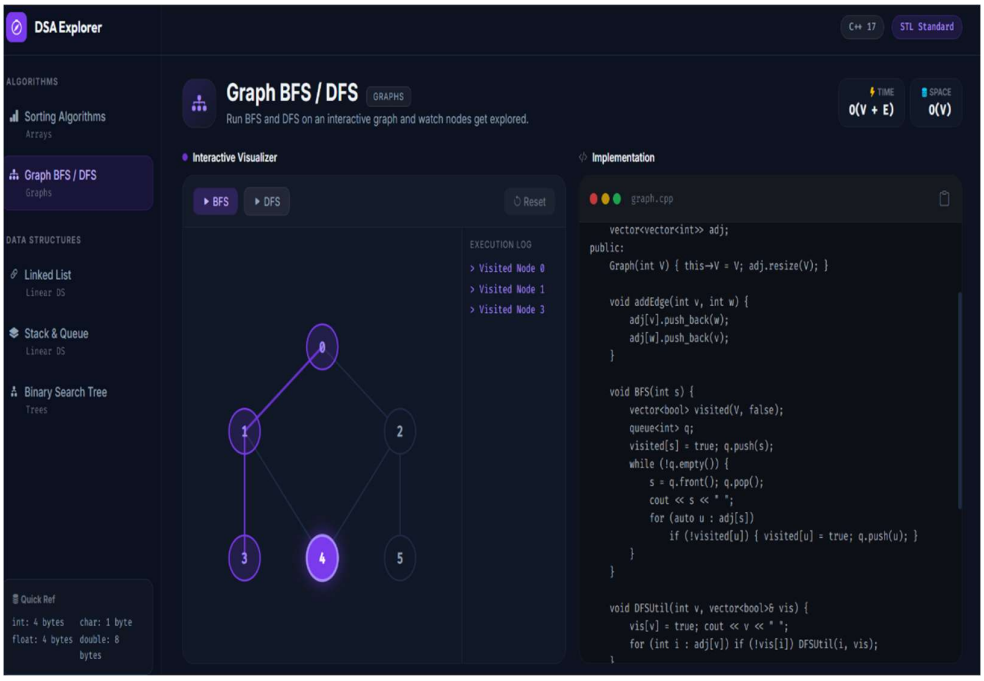


Fig 4. BFS and DFS

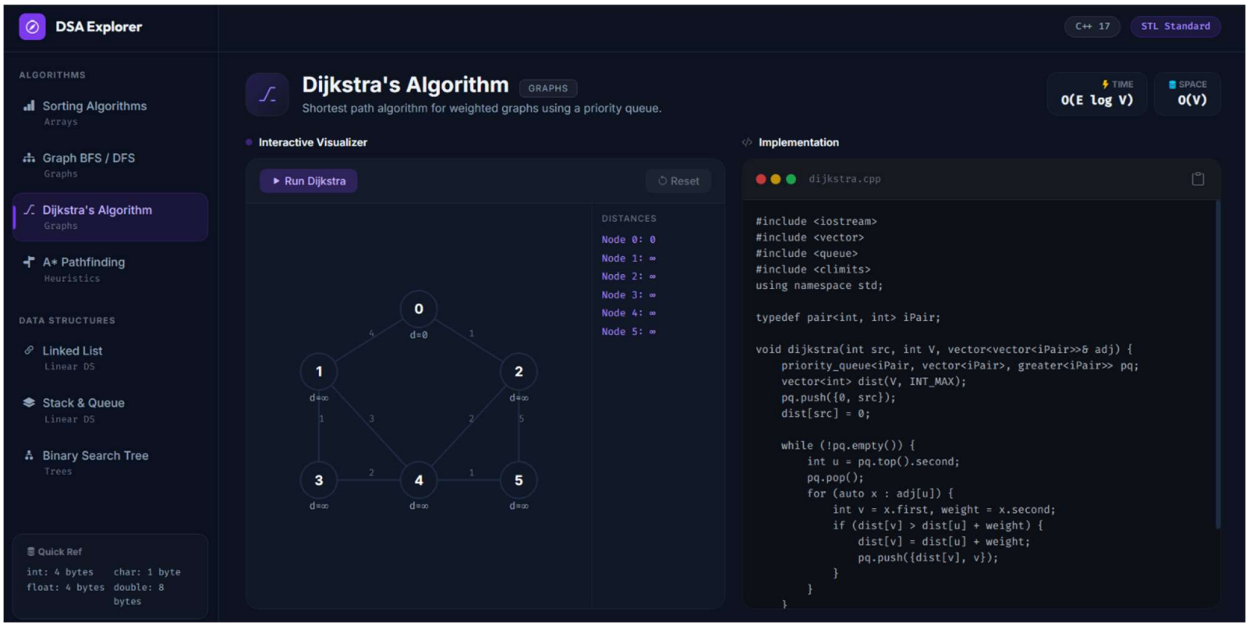


Fig.5 Dijkstra's Algorithm

9. Conclusion:

The **DSA Explorer System** successfully demonstrates how Data Structures and Algorithms can be effectively understood through interactive visualization and modern web technologies. By integrating various algorithms such as sorting techniques, graph traversal methods like BFS and DFS, and operations on data structures like linked lists, stacks, queues, and binary search trees, the system provides a comprehensive learning platform for students and beginners.

The use of a modular and full-stack architecture ensures scalability, flexibility, and easy maintenance, allowing future enhancements such as advanced algorithm visualizations, real-time code execution, and performance comparison tools. The integration of a user-friendly interface built with React, along with backend support using Node.js and Express, enhances usability and accessibility for both technical and non-technical users.

Overall, the DSA Explorer System bridges the gap between theoretical concepts and practical understanding by combining visualization, code representation, and interactive learning. It not only simplifies complex algorithmic concepts but also promotes better problem-solving skills and deeper understanding, making it a valuable educational and practical tool in the field of computer science.

10.Future Work:

The **DSA Explorer System** provides a strong foundation for learning Data Structures and Algorithms, but several enhancements can be made to improve its functionality, interactivity, and real-world applicability:

• **Real-Time Code Execution:**
Integrate an online compiler to allow users to run C++ code directly within the application and see real-time outputs.

• **Advanced Visualization:**
Add more interactive and detailed animations for complex algorithms such as trees, graphs, and dynamic programming.

• **Performance Comparison Tools:**
Provide graphical comparisons of algorithms based on time and space complexity using charts and analytics.

• **Cloud-Based Deployment:**
Deploy the system on cloud platforms to enable multi-user access and online learning from anywhere.

• **Role-Based Access Control:**
Introduce login/signup functionality with roles like student and admin for personalized learning and secure access.

• **Mobile and Web Application Expansion:**
Develop cross-platform mobile applications (Android/iOS) for easy access to algorithm visualizations anytime.

• **Integration with Learning Platforms:**
Connect the system with e-learning platforms or LMS to track user progress and provide quizzes or assessments.

• **Addition of Advanced Algorithms:**
Include more complex topics such as Dijkstra's algorithm, dynamic programming problems, greedy algorithms, and AI-based optimization techniques.

12.GitHub Link:

https://github.com/mrjaiswal122/AlgoVisualizer